Supplementary Materials for 'Gradient-Based Leakage Attacks on GCNNs: A Powerful Threat for Secure Circuit Design'

Appendix

A. Differential Privacy

To mitigate the risk of sensitive information being recovered via gradient inversion, we integrate a differential-privacy mechanism into the gradient extraction process, by perturbing the computed gradients with Gaussian noise. Let

$$G_{\text{true}}^{(l)} = \nabla_{W^{(l)}} \mathcal{L}(\hat{Y}, Y)$$

denote the true gradient of the loss function $\mathcal{L}$ with respect to the model parameters $W^{(l)}$ at layer l, where $\hat{Y}$ and Y are the model's prediction and the ground truth, respectively. We then make the gradient private by adding Gaussian noise:

$$G_{\text{dp}}^{(l)} = G_{\text{true}}^{(l)} + \mathcal{N}(0, \sigma^2 I),$$

where $\mathcal{N}(0, \sigma^2 I)$ denotes Gaussian noise with zero mean and covariance $\sigma^2 I$, and I is the identity matrix. In our experiments, we set $\sigma = 0.1$, i.e., the privacy noise multiplier is 0.1.

Consequently, the reconstruction objective for the gradient leakage attack is modified to minimize the difference between the noisy gradients $G_{\text{dp}}^{(l)}$ and the gradients computed from the dummy input $\tilde{X}$, given by:

$$\min_{\tilde{X}} \sum_{l=0}^{L-1} \left\| \nabla_{W^{(l)}} \mathcal{L}(\tilde{X}) - G_{\text{dp}}^{(l)} \right\|_F^2.$$

In short, by injecting Gaussian noise with $\sigma = 0.1$ into the gradients, we seek to obscure sensitive information while maintaining sufficient utility for model training and evaluation.

B. Gradient Clipping and Perturbation

Here, we implement a defense mechanism that combines gradient clipping with perturbation. Let

$$G_{\text{true}}^{(l)} = \nabla_{W^{(l)}} \mathcal{L}(\hat{Y}, Y)$$

denote the true gradient of the loss $\mathcal{L}$ with respect to the model parameters $W^{(l)}$ at layer l. We first clip these gradients to restrict their magnitudes by enforcing

$$G_{\text{clip}}^{(l)} = \min\left(1, \frac{C}{\|G_{\text{true}}^{(l)}\|_2}\right) G_{\text{true}}^{(l)},$$

where the clipping threshold is set to $C = 1.0$. To further obscure the leaked information, we add Gaussian noise to produce the defended gradient:

$$G_{\text{def}}^{(l)} = G_{\text{clip}}^{(l)} + \mathcal{N}(0, \alpha^2 I),$$

with $\alpha = 0.05$ and I being the identity matrix. The reconstruction objective then becomes

$$\min_{\tilde{X}} \sum_{l=0}^{L-1} \left\| \nabla_{W^{(l)}} \mathcal{L}(\tilde{X}) - G_{\text{def}}^{(l)} \right\|_F^2,$$

where $\tilde{X}$ is the dummy input being optimized. This two-fold defense – clipping gradients to a maximum norm of 1.0 and perturbing them with a noise multiplier of 0.05 – effectively degrades the precision of the gradient signal available to the attacker, leading to higher reconstruction errors and improved privacy protection.

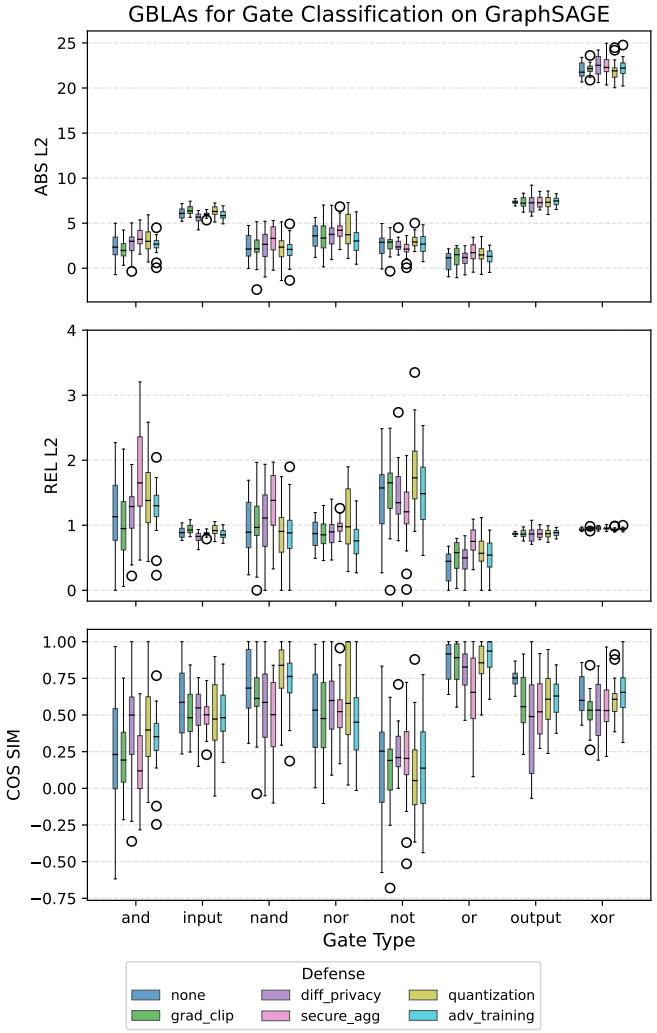

Fig. 1: Gate Classification: Reconstruction errors under different defense mechanism for GraphSAGE.

C. Secure Aggregation Protocols

Here, we apply secure aggregation protocols, wherein gradients are computed and averaged over multiple nodes instead of individual samples. This aggregated approach limits

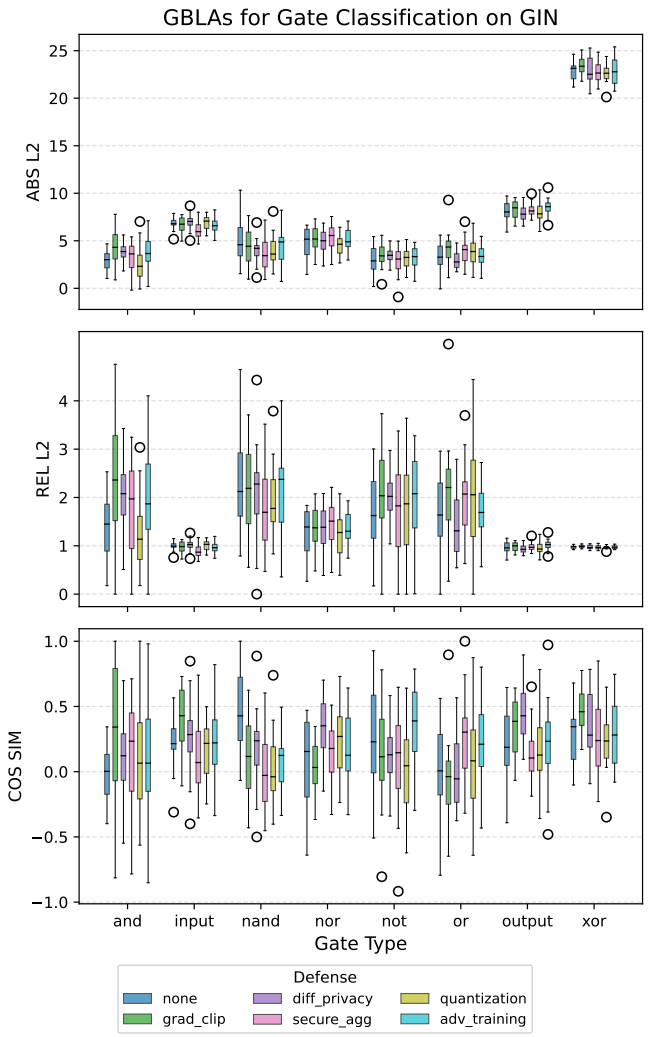

Fig. 2: Gate Classification: Reconstruction errors under different defense mechanism for GIN.

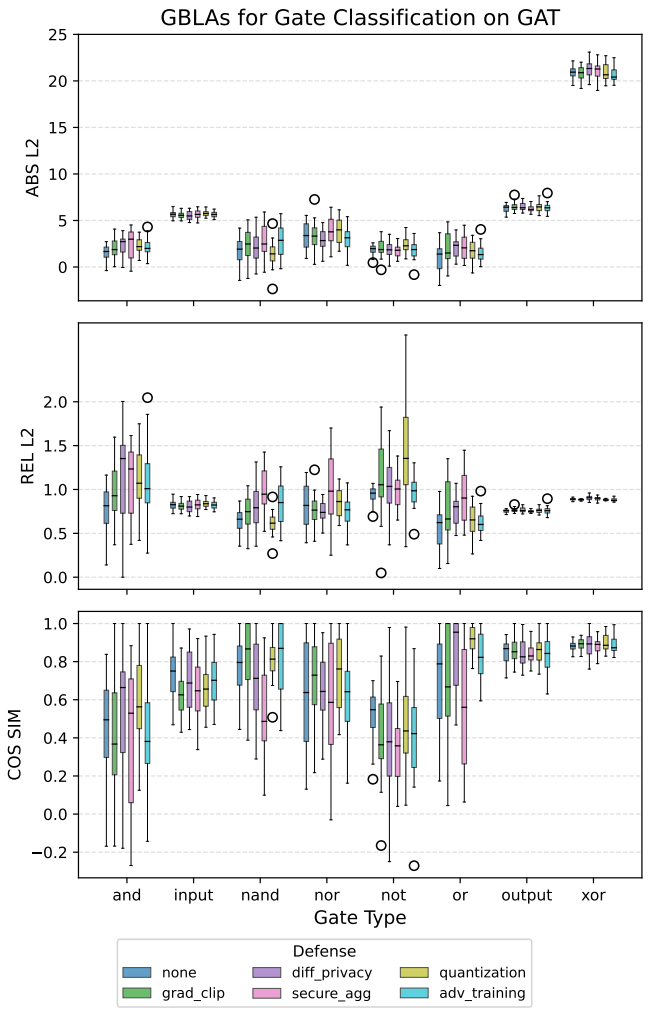

Fig. 3: Gate Classification: Reconstruction errors under different defense mechanism for GAT.

the attacker's ability to reconstruct specific node features while preserving the integrity of model updates.

Let a group of N_{group} nodes be selected from a given class, each with its corresponding feature X_i. The aggregated feature for the group is computed as:

$$X_{\text{agg}} = \frac{1}{N_{\text{group}}} \sum_{i=1}^{N_{\text{group}}} X_i.$$

Similarly, the aggregated gradients for the group are computed using the mean loss over all selected nodes:

$$G_{\text{agg}}^{(l)} = \frac{1}{N_{\text{group}}} \sum_{i=1}^{N_{\text{group}}} \nabla_{W^{(l)}} \mathcal{L}(X_i, Y_i),$$

where $\mathcal{L}(X_i, Y_i)$ represents the loss function for node i.

In our implementation, we set $N_{\text{group}} = 5$ and simulate an attacker attempting to reconstruct X_{agg} rather than individual node features. The reconstruction objective now becomes:

$$\min_{\tilde{X}_{\text{agg}}} \sum_{l=0}^{L-1} \left\| \nabla_{W^{(l)}} \mathcal{L}(\tilde{X}_{\text{agg}}) - G_{\text{agg}}^{(l)} \right\|_F^2.$$

Since the attacker is only provided with the averaged gradient and feature information, individual variations within the group are masked, notably degrading inversion performance.

D. Model Compression and Quantization

Here, we use model compression and quantization techniques to restrict the precision of model parameters, obfuscating critical information that could be exploited by an attacker. This approach involves two key defenses: (1) quantization, where model outputs are discretized to lower precision, and (2) compression through weight pruning, where small-valued parameters are removed to minimize information leakage.

1) Quantization: During the forward pass of the GCNN, numerical outputs are quantized using:

$$X_{\text{quant}} = \frac{\text{round}(X \cdot Q)}{Q},$$

where $Q = 255$ simulates 8-bit fixed-point representation. This transformation limits the granularity of feature updates, reducing inversion fidelity.

2) Compression via Pruning: After backpropagation, small-magnitude model weights are set to zero:

$$W_{\text{compressed}}^{(l)} = \begin{cases} 0, & \text{if } |W^{(l)}| < \tau \\ W^{(l)}, & \text{otherwise} \end{cases}$$

where $\tau = 0.01$ is the pruning threshold. This process eliminates weak connections in the NN, making gradient inversion less effective.

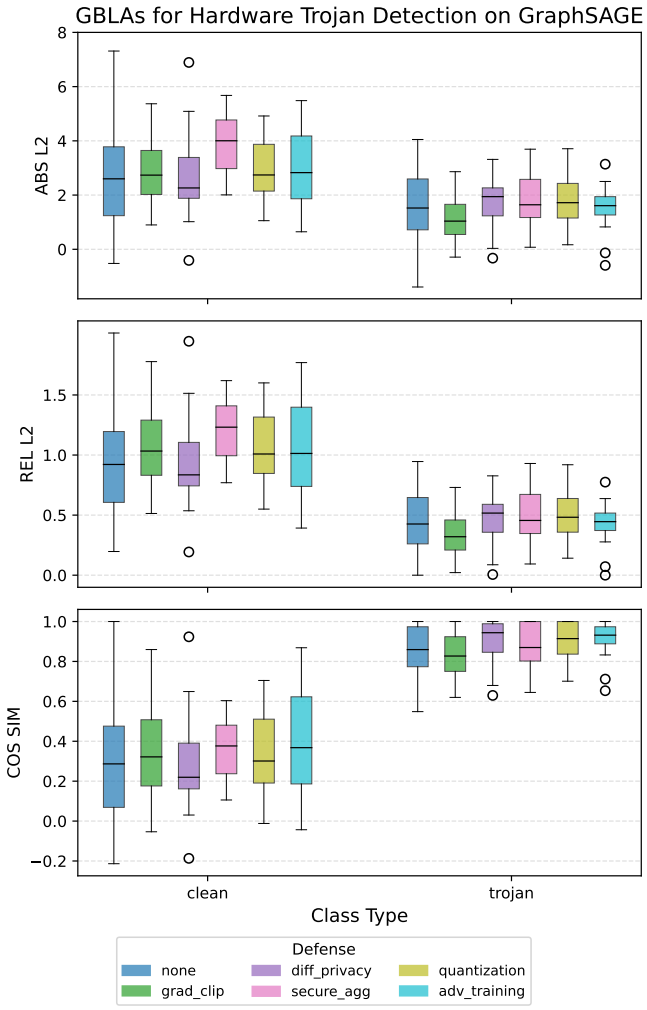

Fig. 4: Hardware Trojan Detection: Reconstruction errors under different defense mechanism for GraphSAGE.

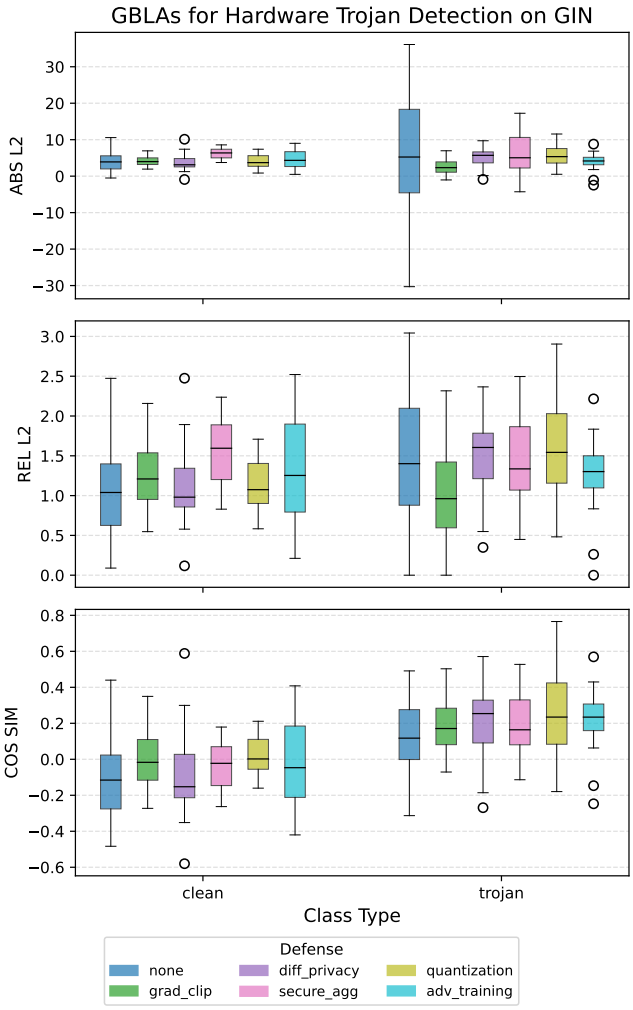

Fig. 5: Hardware Trojan Detection: Reconstruction errors under different defense mechanism for GIN.

E. Adversarial Training

Here, we introduce adversarial perturbations into the model's training process to improve its resilience against GBLAs. By exposing the model to carefully crafted perturbations during training, it learns representations that are less susceptible to inversion attacks.

1) Generating Adversarial Perturbations: To generate adversarial examples, we use the Fast Gradient Sign Method (FGSM), where the perturbation applied to the feature matrix X is computed as:

$$X_{\text{adv}} = X + \epsilon \cdot \text{sign}(\nabla_X \mathcal{L}(X, Y)),$$

where ϵ is the perturbation strength (set to 0.1 in our experiments), $\mathcal{L}(X, Y)$ is the loss function, and $\nabla_X \mathcal{L}$ represents the gradient of the loss with respect to the input features.

2) Adversarial Training Procedure: During training, instead of using the original features, we compute adversarial perturbations and feed the model with X_{adv}, reinforcing its ability to generalize across perturbed data:

$$\min_W \sum_{i=1}^{N} \mathcal{L}(f_W(X_{\text{adv},i}), Y_i).$$

This forces the model to optimize for robustness rather than merely fitting the clean training data.

The box plots for different metrics for both the gate classification and the hardware Trojan detection on GraphSAGE, GIN, and GAT models are shown in Figs. 1, 2, 3, 4, 5, and 6 respectively.

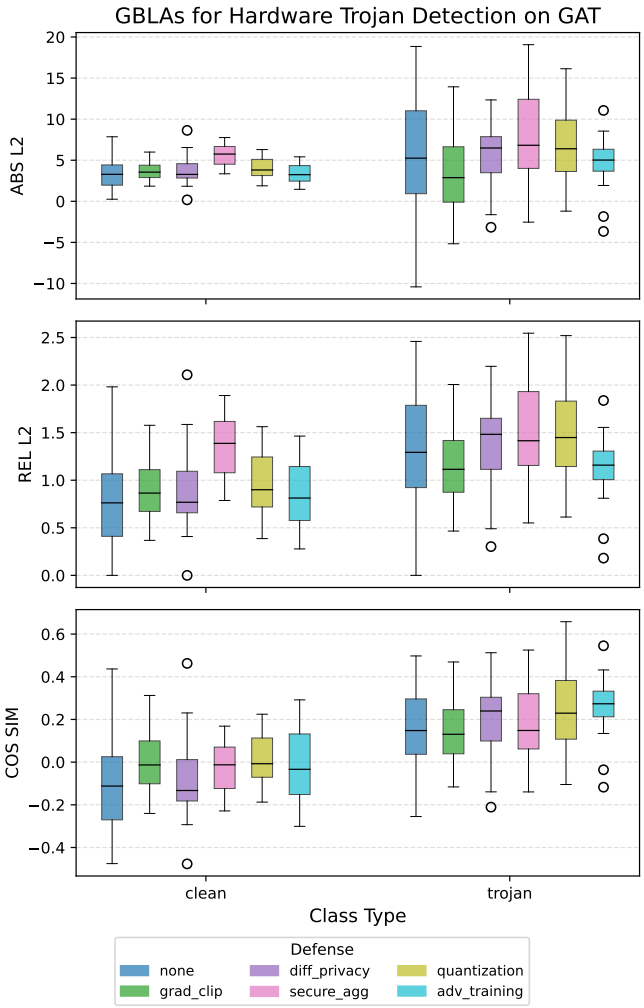

Fig. 6: Hardware Trojan Detection: Reconstruction errors under different defense mechanism for GAT.